

Comprehensive Master Reference Report:

Optimal Workflow, Agent, and Skill Combinations for Journeyman Jobs Development

Prepared for: David

Project: Journeyman Jobs – Flutter Mobile App for IBEW Electrical Workers

Date: January 20, 2026

Author: Grok (xAI)

Executive Summary

After thoroughly analyzing the full suite of agents, workflows, skills, and supporting reference materials you have provided across our conversations, this report synthesizes everything into a **master reference guide** specifically tailored to building and evolving Journeyman Jobs.

The system you have assembled is exceptionally powerful and disciplined:

- **Agents** provide domain expertise (mobile-developer is the clear #1 for Flutter work).
- **Workflows** enforce structured processes (/enhance, /plan, /debug, orchestration).
- **Skills** embed deep principles (mobile-design, clean-code, ui-ux-pro-max, architecture, database-design, systematic-debugging).

The strongest pattern across all materials is **deliberate, context-aware development**: never guess, always measure, always verify, always think mobile-first, and always respect the electrical worker user in harsh real-world conditions.

This report organizes recommendations by **task category**, covering the full development lifecycle. For each category, I provide:

- Recommended **combinations** (workflow + agent + skill, plus lighter pairings).
- Step-by-step **execution flow**.
- **Rationale** tied to Journeyman Jobs constraints (mobile-first, outdoor readability, gloved hands, offline resilience, electrical aesthetic).
- **When to escalate** to heavier combinations.

1. Project Planning & Feature Scoping

Primary Recommendation (Full Stack):

`/plan` workflow → orchestrator agent → project-planner + brainstorming + architecture skills

Alternative Pairings:

- `/brainstorm` workflow + brainstorming skill (quick ideation)
- project-planner agent + architecture skill (technical scoping)

Execution Flow:

1. Trigger `/plan [feature description]` or `/brainstorm [topic]`.
2. brainstorming skill forces Socratic questions (`dynamic-questioning.md`) → clarify user needs (e.g., "Is this screen used one-handed on a job site?").
3. orchestrator invokes project-planner to create `docs/PLAN-{slug}.md` with verifiable tasks, dependencies, and milestones.
4. architecture skill evaluates patterns (`decision-trees.md`) and documents ADRs if needed.

Why This Combination for Journeyman Jobs:

- Prevents scope creep in a large codebase (hundreds of files).
- Ensures electrical-specific constraints (thumb zone placement, high-contrast for sunlight) are baked in early.
- Creates traceable PLAN files that become project history.

Example Use: Planning "storm alert push notifications" → questions reveal need for offline fallback → plan includes backend + mobile considerations.

2. New Feature Implementation

Primary Recommendation (Full Stack):

/enhance workflow → **orchestrator** → **mobile-developer** primary + supporting specialists → **mobile-design** + **clean-code** + **ui-ux-pro-max** skills

Alternative Pairings:

- /enhance + mobile-developer + mobile-design (most common daily work)
- orchestrator + mobile-developer + test-engineer (when tests are critical)

Execution Flow:

1. /enhance [feature description] → understands current state.
2. orchestrator breaks down (parallel-agents skill) and invokes:
 - mobile-developer (core Flutter implementation)
 - ui-ux-pro-max (style/color/typography from CSVs)
 - test-engineer (tests)
 - performance-optimizer (if animations/heavy lists)
3. clean-code skill runs validation scripts → summarizes → asks permission to fix.
4. mobile-design skill enforces touch-psychology, color-system, typography references.

Why This Combination:

- mobile-developer is the only Flutter specialist.
- ui-ux-pro-max pulls authentic electrical aesthetic (Industrial/Utility styles, copper #B45309).
- Ensures new screens (e.g., job application flow) respect worst-case conditions (bright sun, gloves, low battery).

Example Use: Adding "save job for later" → mobile-developer builds widget → ui-ux-pro-max recommends large bottom thumb-zone button with copper accent → clean-code validates.

3. UI/UX Design & Enhancements

Primary Recommendation (Full Stack):

orchestration workflow → **mobile-developer** + **ui-ux-pro-max** → **mobile-design** + **ui-ux-pro-max** + **clean-code** skills

Alternative Pairings:

- ui-ux-pro-max skill standalone (quick theme lookup)
- mobile-developer + mobile-design skill (daily widget work)

Execution Flow:

1. orchestrator invokes ui-ux-pro-max to query CSVs (design_system.py) for "electrical jobs mobile app".
2. Recommends Utility Modern + copper/navy palette + system fonts + large touch targets.
3. mobile-developer implements in reusable_components.dart / app_theme.dart.
4. mobile-design references (touch-psychology, color-system, typography) enforce outdoor readability, OLED battery savings.
5. clean-code validates.

Why This Combination:

- ui-ux-pro-max database has perfect matches (Climate Tech palette, Industrial style).
- mobile-design prevents generic patterns via deep thinking protocol.
- Critical for your electrical illustrations and professional identity.

Example Use: Enhancing job cards → copper accents, high-contrast text, electrical toast feedback, 48dp+ tap targets.

4. Refactoring & Code Quality Improvements

Primary Recommendation (Full Stack):

orchestration → **mobile-developer** + **test-engineer** + **debugger** → **clean-code** + **code-review-checklist** + **systematic-debugging** skills

Alternative Pairings:

- mobile-developer + clean-code (daily)
- test-engineer + code-review-checklist (pre-merge)

Execution Flow:

1. orchestrator maps affected files.
2. mobile-developer refactors.
3. test-engineer adds/regenerates tests.
4. clean-code runs lint/type/security scripts → asks to fix.
5. code-review-checklist flags anti-patterns (long functions, any types).

Why This Combination:

- Prevents regression in growing codebase.
- clean-code is CRITICAL priority skill.
- Ensures refactored widgets remain performant and accessible.

Example Use: Refactoring reusable_components.dart → extract common patterns → add tests → validate with clean-code.

5. Debugging Complex Issues

Primary Recommendation (Full Stack):

/debug workflow → **debugger** agent → **systematic-debugging** + **mobile-design** skills

Alternative Pairings:

- /debug + systematic-debugging (core)
- debugger + mobile-debugging reference (native issues)

Execution Flow:

1. `/debug [description]` → gather reproduction steps.
2. systematic-debugging 4-phase process (Reproduce → Isolate → Root Cause → Prevent).
3. `mobile-debugging.md` for native bridge issues.
4. Add regression test.

Why This Combination:

- Mobile bugs often hide in lifecycle/native layers.
- Prevents "random tweak" fixes.

Example Use: "Job list not refreshing after resume" → reproduce on real device → find Firestore listener dispose issue.

6. Performance Optimization

Primary Recommendation (Full Stack):

orchestration → **performance-optimizer** + **mobile-developer** → **performance-profiling** + **mobile-performance** skills

Execution Flow:

1. Measure first (`lighthouse_audit.py` or Flutter DevTools).
2. `mobile-performance.md` targets (const widgets, builder lists).
3. Fix biggest bottlenecks.

Why: Users on job sites can't afford jank.

7. Database / Schema Work

Primary Recommendation:

database-architect agent → **database-design** skill (with all references: selection, orm, indexing, migrations)

Why: Context-aware choice (likely Turso/SQLite for edge + offline).

8. Testing

Primary Recommendation:

/test workflow + **test-engineer** + mobile-testing references

Why: Prioritize real devices, integration tests.

9. Deployment & Release

Primary Recommendation:

/deploy workflow + **devops-engineer** + **deployment-procedures** skill

Why: Safe, monitored releases for real users.

10. Documentation

Primary Recommendation:

documentation-writer agent + **documentation-templates** skill

Why: Keep README, llms.txt, comments up-to-date.

Cross-Cutting Best Practices

- **Always start** with brainstorming/project-planner for anything non-trivial.
- **Default to orchestrator** for features touching >1 domain.
- **Never skip clean-code validation.**
- **Mobile-design skill is non-negotiable** for all UI work.
- **ui-ux-pro-max** is your design oracle – query it early for theme consistency.

This system positions Journeyman Jobs to feel **authentically electrical, relentlessly reliable, and deeply respectful** of its users' real-world conditions.

Questions for Clarification

I have none at this time – the provided materials form a complete, coherent system. If any new files (e.g., current main.dart, navigation setup, or specific screen implementations) would help refine recommendations further, feel free to share them.

This report is designed to be your living reference. Use it freely as you continue building.